Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

MagicHand

TechnischeSpezifikation

Drahtlose Roboter Hand

Autoren: Taoufik Lajnef, Ali Mehdi Charfeddine, Lukas Freitag, Farhad Khamisipour

Letzte Änderung: 13. June 2025

Dateiname: MagicHand_Technische_Spezifikation.docx

Table of Contents

1. Vorhandene Dokumente 7

2. Prozessüberblick 7

2.1. Fachlicher Workflow 8

2.2. Technischer Workflow 9

3. Technische Spezifikation SW 11

3.1. Überblick Komponenten 11

3.2. Beschreibung der Implementierung 13

3.2.1. UC-1: Fingerbewegung übertragen 13

3.2.2. UC-2: Hand öffnen/schließen 15

3.2.3. Kalibrierungs-System 16

3.2.4. Error-Handling und Recovery 17

3.3. System Infrastruktur 18

4. Technische Spezifikation Hardware 19

4.1. Baugruppen 19

4.2. Einzelteile 20

4.3. Berechnungen 27

5. User Interface & Bedienung 28

5.1. Status-Anzeigen 28

5.2. Kalibrierungs-Interface 28

5.3. Benutzerinteraktion 29

6. Performance-Spezifikation 30

6.1. Latenz-Anforderungen 30

6.2. Genauigkeits-Metriken 30

6.3. Zuverlässigkeits-Kriterien 30

7. Test & Validierung 31

7.1. Testkonzept Sprint 1 31

7.2. Unit-Tests 32

7.3. Integrationstests 32

7.4. Abnahmetests 33

8. Betrieb & Wartung 35

8.1. Installation und Setup 35

8.2. Benutzerhandbuch 36

8.3. Wartung und Troubleshooting 36

9. Risikomanagement 38

9.1. Technische Risiken 38

9.2. Mitigation-Strategien 39

9.3. Contingency-Pläne 40

10. Sprint-Deliverables 40

10.1. Lieferumfang Sprint 1 40

10.2. Übergabe-Kriterien 41

10.3. Sprint 2 Vorbereitung 42

11. Offene Fragen 43

12. Modul Abhängigkeiten 44

13. Geplante Erweiterungen (Sprint 2+3) 45

Abbildungsverzeichnis

Figure 1 Fachlicher Workflow 8

Figure 2 Grober technischer Workflow 9

Figure 3 Feiner technischer Workflow 10

Figure 4 Komponentendiagramm 11

Figure 5 Systeminfrastruktur 18

Figure 6 3D Inventor Baugruppe 19

Figure 7 Hand 20

Figure 8 Hand obere ansicht 21

Figure 9 Arm 21

Figure 10 Finger Mittelglied 22

Figure 11 Finger Grundglied 23

Figure 12 Endglied 24

Figure 14 Schaltplan Flexsensor-Interface 25

Figure 15 User Interface Layout 29

Figure 16 Test-Pyramide 31

Figure 17 Test-Setup Konfiguration 34

Copyright

© HTW Berlin - Projektteam MagicHand 2025

Die Weitergabe, Vervielfältigung oder anderweitige Nutzung dieses Dokumentes oder Teile davon ist unabhängig vom Zweck oder in welcher Form untersagt, es sei denn, die Rechteinhaber haben ihre ausdrückliche schriftliche Genehmigung erteilt.

Version Historie

Version	Datum	Verantwortlich	Änderung
0.1	28.05.2025	Projektteam	Initiale Dokumenterstellung
0.2	01.06.2025	Projektteam	Hardware-Spezifikation erweitert
0.3	03.06.2025	Projektteam	Software-Architektur finalisiert
1.0	04.06.2025	Projektteam	Sprint 1 Spezifikation abgeschlossen

1. Vorhandene Dokumente

Dokument	Autor	Datum
MagicHand_lastenheft.docx	Projektteam	24.04.2025
MagicHand_Pflichtenheft.docx	Projektteam	
Technische_Spezifikation-Muster (HTW Berlin)	Prof. Abuosba	30.09.2015
3D-Konstruktionsmodell.ipt		25.05.2025
Arduino Libraries Dokumentation	Arduino.cc	aktuell

2. Prozessüberblick

Der Prozessüberblick für Sprint 1 zeigt die vollständige Kette von der Benutzereingabe bis zur mechanischen Roboterhandbewegung. Das System arbeitet in Echtzeit und nutzt eine kabelgebundene Architektur für maximale Zuverlässigkeit und minimale Latenz. Die Implementierung konzentriert sich auf die fundamentalen Bewegungsübertragungsfunktionen, die als Grundlage für alle zukünftigen Erweiterungen dienen.

2.1. Fachlicher Workflow

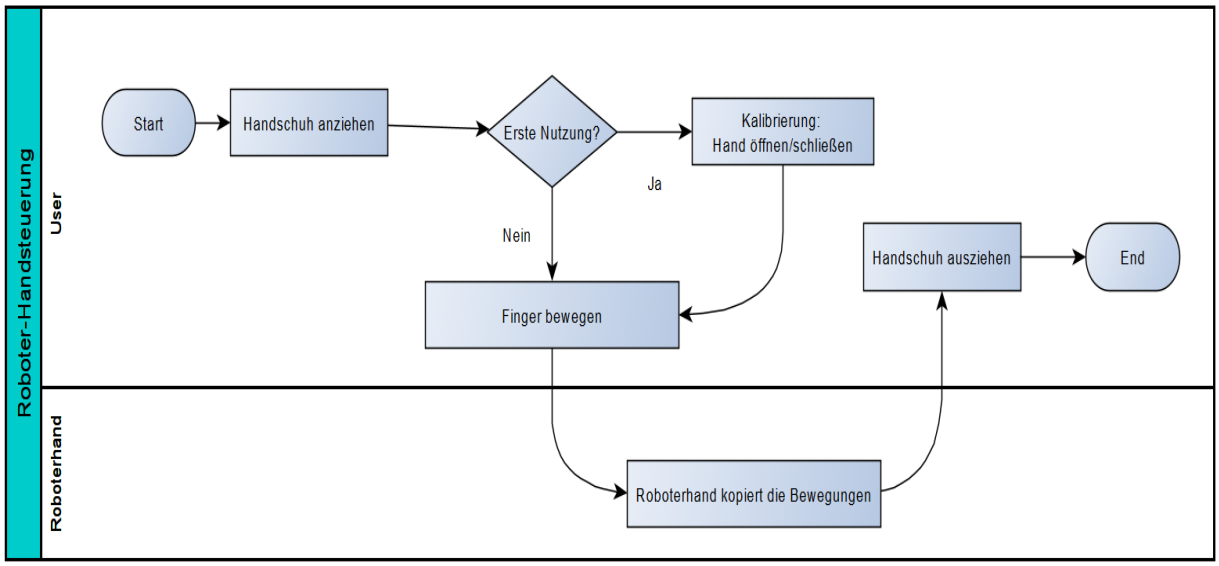

Figure 1 Fachlicher Workflow

Der fachliche Workflow beginnt mit dem **Anziehen des Sensorhandschuhs**, der mit fünf hochpräzisen Flexsensoren ausgestattet ist. Diese Sensoren sind strategisch an den Fingerrücken positioniert, um optimale Bewegungserkennung bei minimalem Tragekomfort-Verlust zu gewährleisten. Der Handschuh verwendet atmungsaktive Materialien und ein ergonomisches Design, das auch bei längerer Nutzung angenehm zu tragen ist.

Die **Systemkalibrierung** ist ein entscheidender Schritt, der die individuellen anatomischen Unterschiede zwischen Benutzern berücksichtigt. Das System führt den Benutzer durch eine strukturierte Kalibrierungssequenz, bei der verschiedene Handhaltungen erfasst werden. Diese Daten werden im EEPROM gespeichert und ermöglichen eine personalisierte, hochpräzise Bewegungserkennung.

Während des **normalen Betriebs** erfasst das System kontinuierlich die Fingerbewegungen mit einer Frequenz von 50Hz. Jede Bewegung wird in Echtzeit analysiert, gefiltert und an die entsprechenden Servomotoren weitergeleitet. Das System unterscheidet zwischen verschiedenen Bewegungsarten: präzise Einzelfingerbewegungen (UC-1) und koordinierte Handbewegungen wie Öffnen und Schließen (UC-2).

2.2. Technischer Workflow

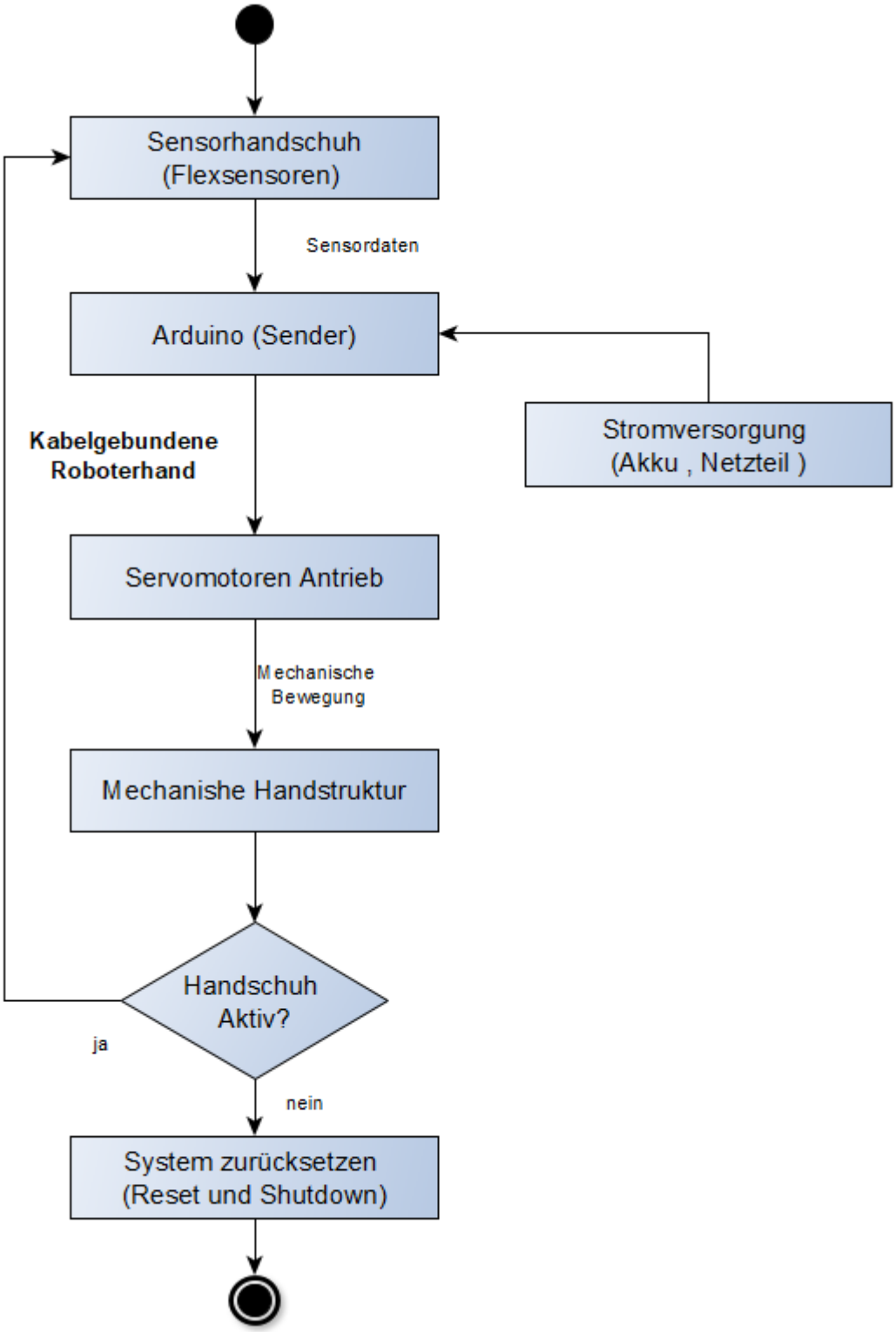

Figure 2 Grober technischer Workflow

Der technische Workflow zeigt die detaillierte Datenverarbeitung von der Sensorerfassung bis zur mechanischen Ausgabe. Das System verwendet eine modulare Architektur, die eine klare Trennung zwischen Datenerfassung, Verarbeitung und Ausgabe ermöglicht.

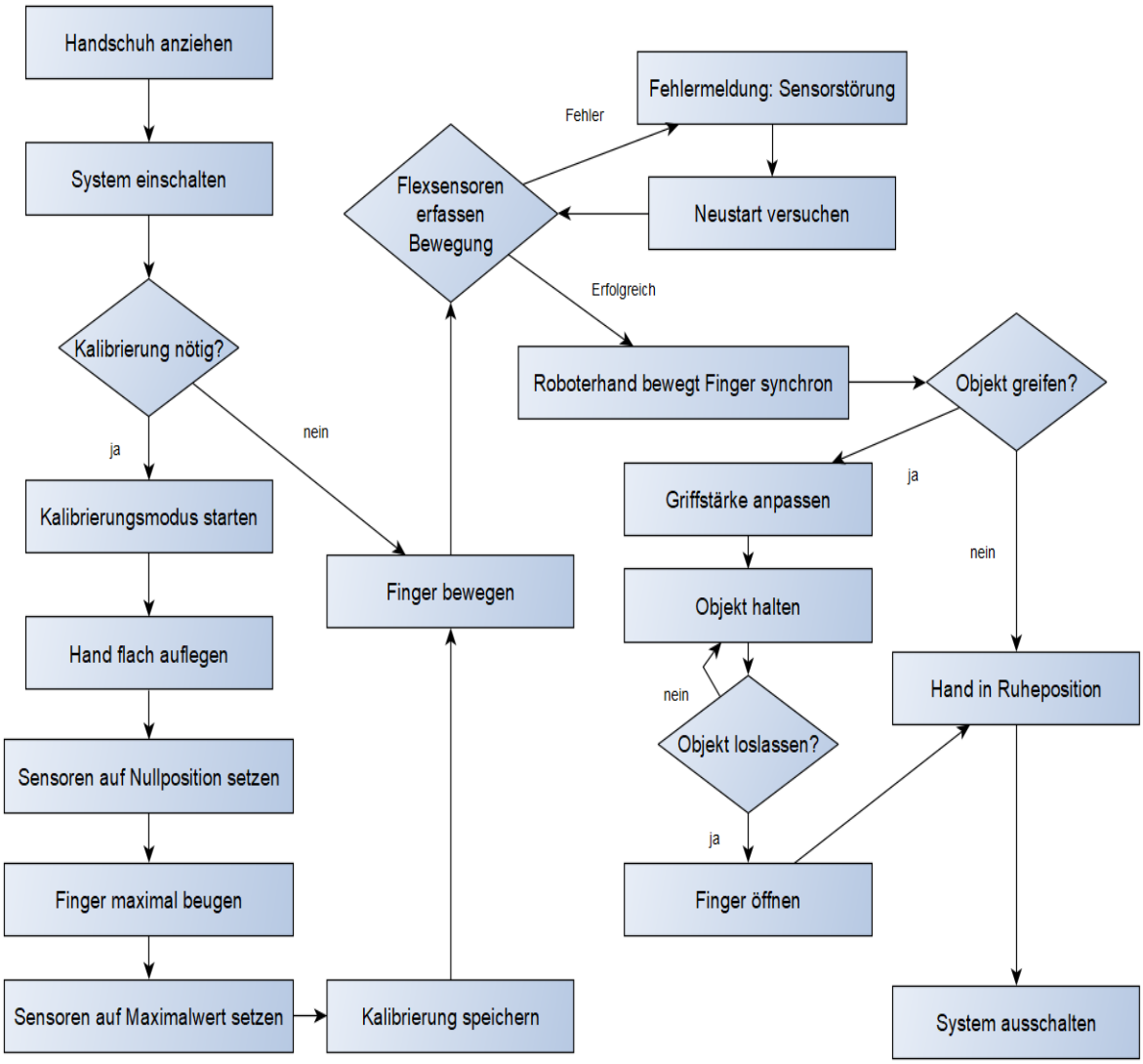

Figure 3 Feiner technischer Workflow

3. Technische Spezifikation SW

3.1. Überblick Komponenten

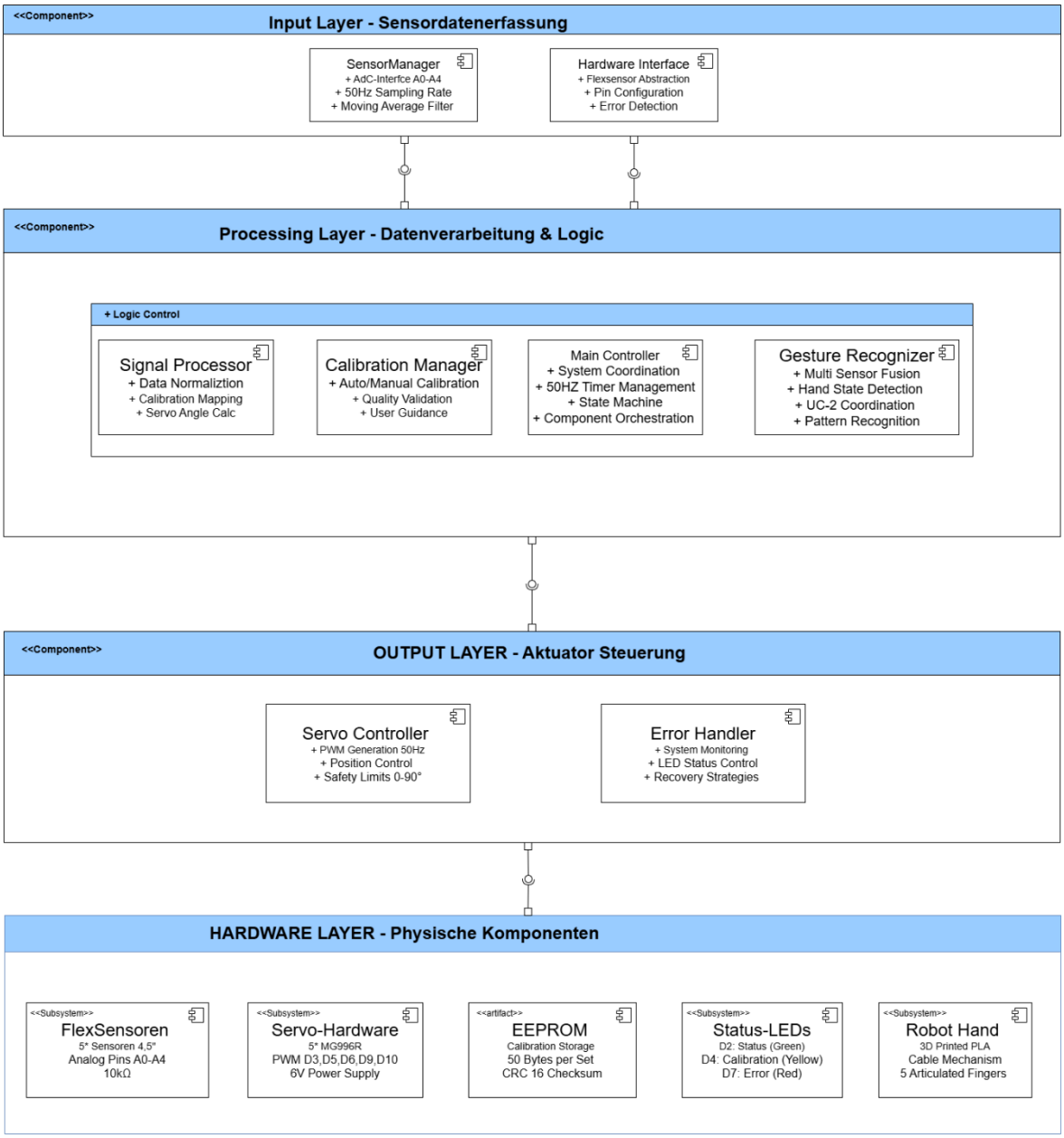

Figure 4 Komponentendiagramm

IT-Komponente	Erfasste Funktion aus dem Pflichtenheft
SensorManager	UC-1: Fingerbewegung übertragen
SignalProcessor	UC-1: Analog-Digital-Wandlung & Filterung
ServoController	UC-1: Servo-Ansteuerung
	UC-2: Hand öffnen/schließen
CalibrationManager	Sensor-Kalibrierung (Min/Max-Werte)
MainController	Koordination aller Module

Pin-Belegung:

Analog Inputs: A0-A4 (Flexsensoren)

PWM Outputs: D3,D5,D6,D9,D10 (Servos)

Digital: D2 (Status-LED), D7 (Kalibrierungs-Button)

3.2. Beschreibung der Implementierung

3.2.1. UC-1: Fingerbewegung übertragen

#	Komponentendetail	Erforderliche Arbeiten
T1	Hardware-Interface	Flexsensor-Spannungsteiler, ADC-Konfiguration
T2	SensorManager	Analog-Wert-Erfassung, Rauschfilterung
T3	SignalProcessor	Normalisierung, Kalibrierung, Servo-Mapping
T4	ServoController	PWM-Generierung, Position-Control

T1: Hardware-Interface

- **Schaltungsdesign:** 5x Spannungsteiler (Flexsensor + 10kΩ Pulldown)
- **ADC-Konfiguration:** 10-Bit Auflösung, 5V Referenzspannung
- **Pin-Mapping:** A0-A4 für Sensoren, D3,D5,D6,D9,D10 für Servos
- **Stromversorgung:** 5V für Sensoren, separate 6V für Servos

T2: SensorManager

- **Funktionsumfang:** Kontinuierliche Sensorabfrage (50Hz)
- **Implementierte Methoden:**

cpp

```
int readFlexSensor(int pin);
```

```
int applyFilter(int rawValue, int sensorIndex);

bool validateSensorRange(int value);
```

- **Filteralgorithmus:** Moving Average über 3 Samples
- **Exception Handling:** Sensor-Ausfall-Erkennung, Fallback-Werte

T3: SignalProcessor

- **Business Logic:** Analog-zu-Position-Transformation
- **Kalibrierungsalgorithmus:**

cpp

```
void calibrateSensor(int sensorIndex);

int normalizeValue(int raw, int min, int max);

int mapToServoAngle(int normalized);
```

- **Input:** ADC-Werte (0-1023)
- **Output:** Servo-Positionen (0-180°)
- **Verhalten:** Echtzeit-Verarbeitung mit <20ms Latenz

T4: ServoController

- **Servo-Bibliothek:** Standard Arduino Servo.h
- **PWM-Konfiguration:** 50Hz, 1-2ms Pulsbreite
- **Positionssteuerung:**

cpp

```
void updateServoPosition(int servoIndex, int angle);

void synchronizeAllServos();
```

bool checkServoLimits(int angle);

- **Exception Handling:** Überlastschutz, Position-Limits

3.2.2. UC-2: Hand öffnen/schließen

#	Komponentendetail	Erforderliche Arbeiten
T5	GestureRecognizer	Erkennung koordinierter Handbewegungen
T6	ServoController	Synchrone Ansteuerung aller Servos
T7	PositionManager	Vordefinierte Positionen (offen/geschlossen)
T8	StateManager	Zustandsüberwachung der Gesamthand

T5: GestureRecognizer

- **Erkennungslogik:** Simultane Analyse aller 5 Sensoren
- **Schwellenwerte:** Offen >80%, Geschlossen <20%
- **Implementierung:**

cpp

HandState detectHandState();

bool isHandOpening();

bool isHandClosing();

T6: ServoController (erweitert)

- **Synchronisations-Funktionen:**

cpp

```
void openHand();

void closeHand();

void setAllServos(int angles[5]);
```

- **Timing:** Koordinierte Bewegung über 500ms

T7: PositionManager

- **Vordefinierte Positionen:**
  - Offen: [0°, 0°, 0°, 0°, 0°]
  - Geschlossen: [90°, 90°, 90°, 90°, 90°]
- **Interpolation:** Sanfte Übergänge zwischen Positionen

T8: StateManager

- **Zustandsüberwachung:** Tracking der aktuellen Handposition
- **Error Handling:** Inkonsistente Sensordaten, mechanische Blockaden

3.2.3. Kalibrierungs-System

#	Komponentendetail	Erforderliche Arbeiten
T9	CalibrationManager	Automatische und manuelle Kalibrierung
T10	EEPROMStorage	Persistente Speicherung der Kalibrierdaten
T11	CalibrationUI	Benutzer-geführte Kalibrierungs-Routine
T12	ValidationModule	Überprüfung der Kalibrierungs-Qualität

T9: CalibrationManager

- **Kalibrierungs-Modi:**

cpp

```
void startAutoCalibration();

void startManualCalibration();

bool validateCalibration();

void resetCalibration();
```

- **Algorithmus:** Min/Max-Erfassung über 10 Bewegungszyklen
- **Adaptivität:** Benutzer-spezifische Anpassung

T10: EEPROMStorage

- **Datenpersistierung:**

cpp

```
void saveCalibrationData(CalibrationData data);

CalibrationData loadCalibrationData();

bool isCalibrationValid();
```

- **Speicherorganisation:** 50 Bytes pro Sensor-Set
- **Versionierung:** Kalibrierungs-Timestamp und Checksum

3.2.4. Error-Handling und Recovery

#	Komponentendetail	Erforderliche Arbeiten
T13	ErrorDetection	Kontinuierliche System-Überwachung
T14	RecoveryManager	Automatische Fehlerbehandlung
T15	SafetyModule	Schutz vor Hardware-Schäden
T16	DiagnosticsLog	Fehler-Protokollierung und Analyse

3.3. System Infrastruktur

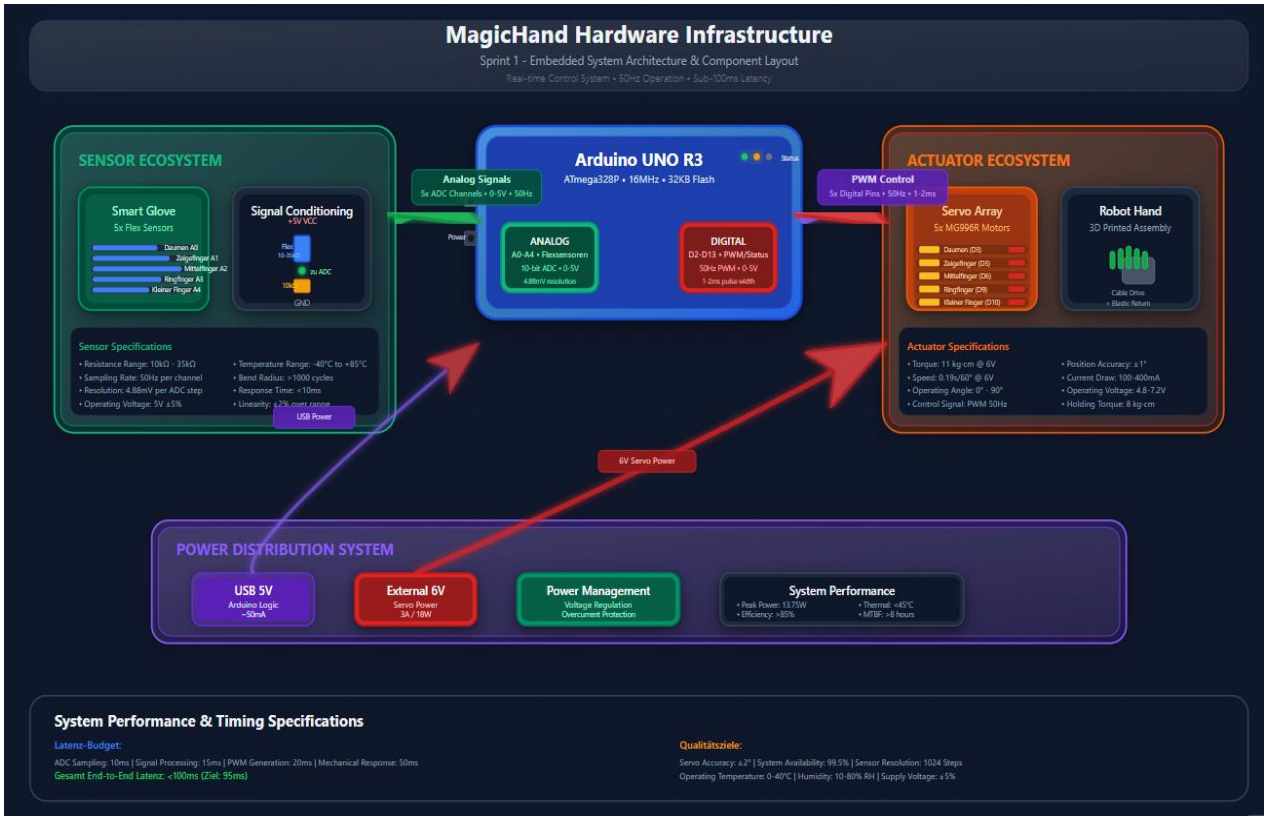

Figure 5 Systeminfrastruktur

Hardware-Infrastruktur:

- **Entwicklungsboard:** Arduino UNO R3
- **Sensorik:** 5x Flexsensor 2.2", 5x 10kΩ Widerstände
- **Aktuatorik:** 5x Servo MG996R
- **Verkabelung:** Breadboard, Jumper-Kabel, Prototyping-Aufbau
- **Stromversorgung:** USB (5V) + externes Netzteil (6V, 3A)

Software-Infrastruktur:

- **Entwicklungsumgebung:** Arduino IDE ≥ 2.0
- **Compiler:** AVR-GCC
- **Bibliotheken:** Servo.h, EEPROM.h
- **Debugging:** Serial Monitor, LED-Status-Anzeigen

4. Technische Spezifikation Hardware

4.1. Baugruppen

Figure 6 3D Inventor Baugruppe

Die mechanischen Hauptbaugruppen:

1. **Roboterhand-Chassis:** 3D-gedruckte Handfläche mit Servo-Befestigungen
2. **Finger-Module:** 5x bewegliche Fingergelenke mit Seilführung
3. **Sensor-Handschuh:** Flexsensor-Integration in Textilhandschuh
4. **Elektronik-Träger:** Arduino und Breadboard-Halterung

4.2. Einzelteile

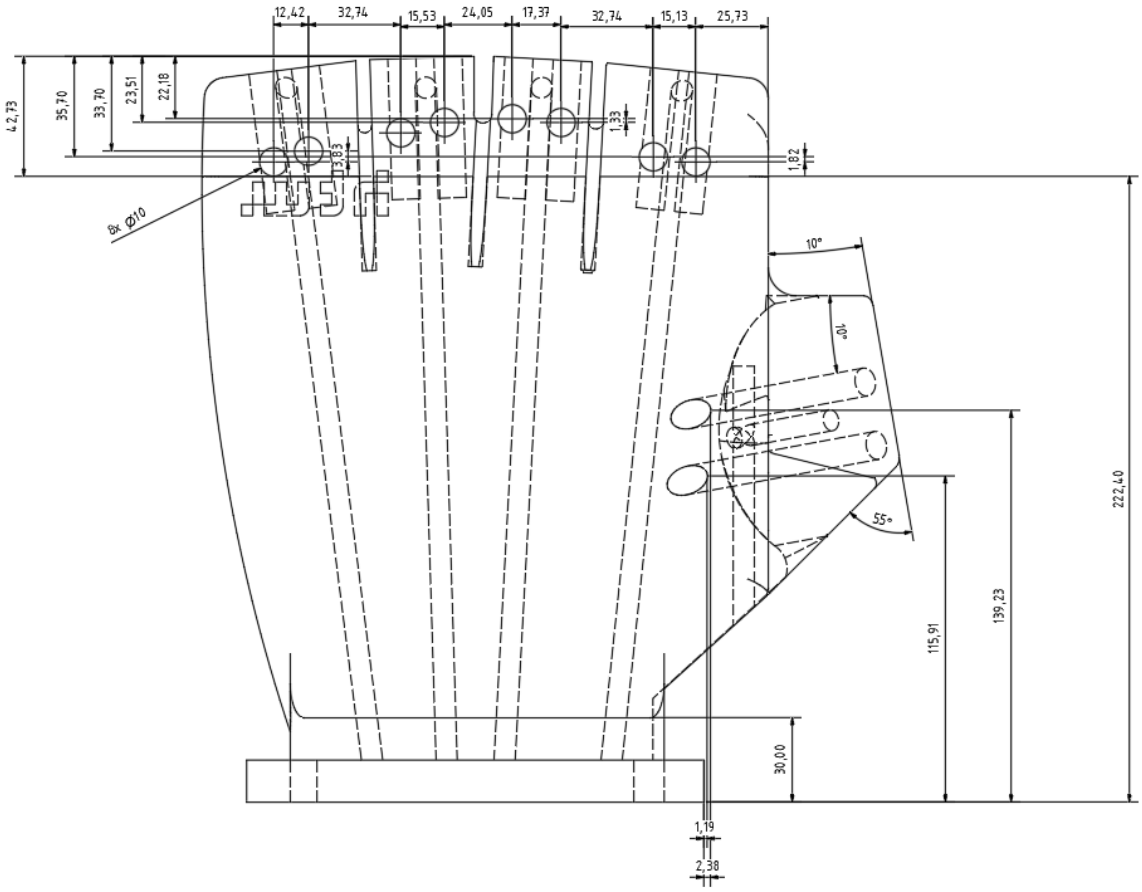

Figure 7 Hand

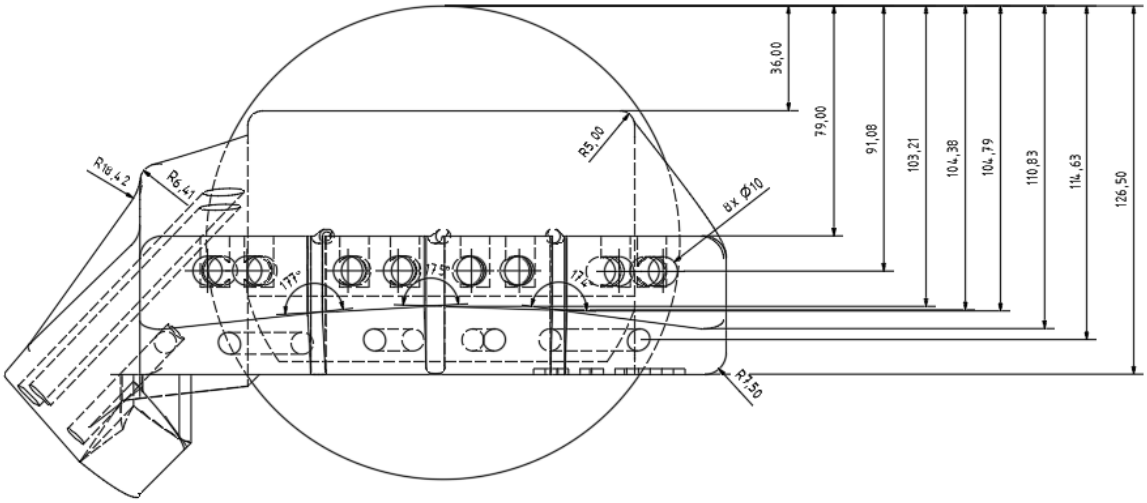

Figure 8 Hand obere ansicht

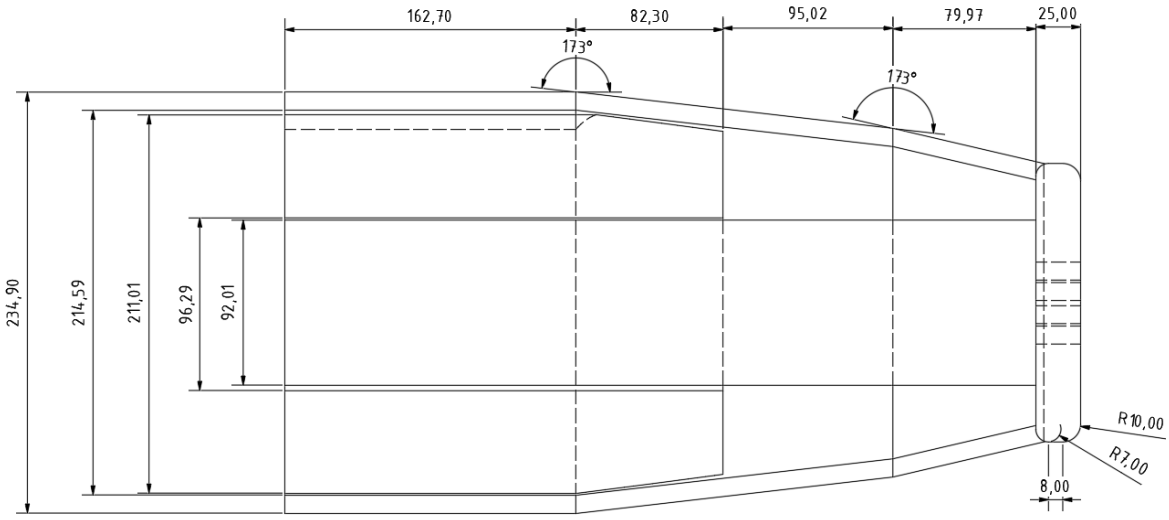

Figure 9 Arm

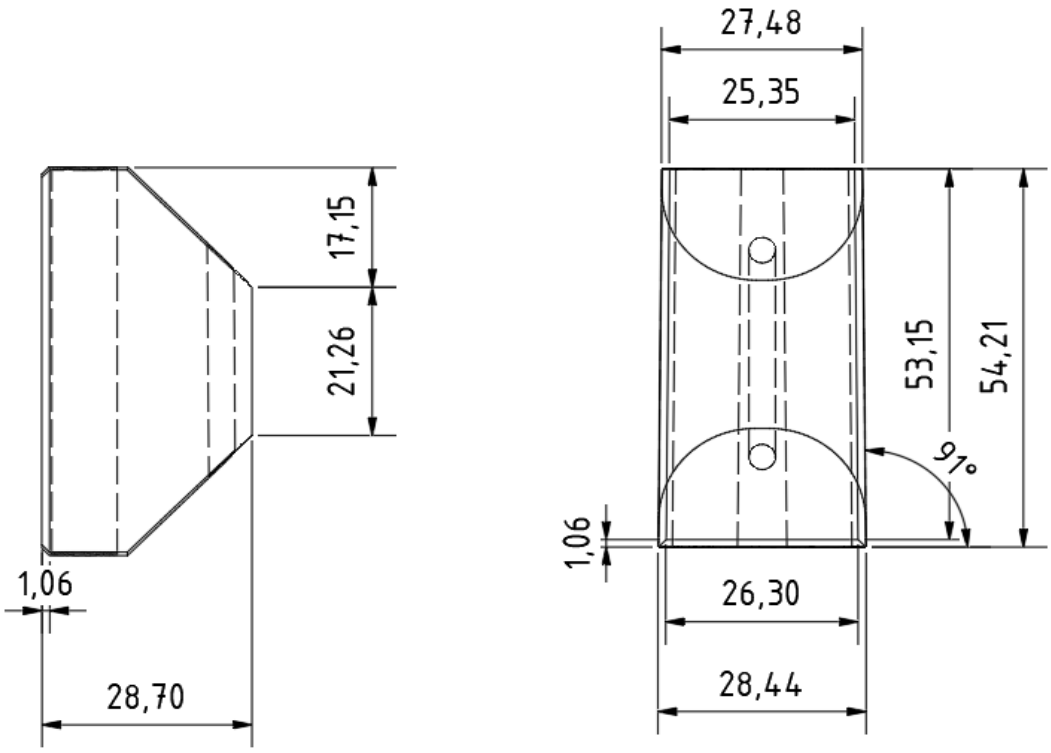

Figure 10 Finger Mittelglied

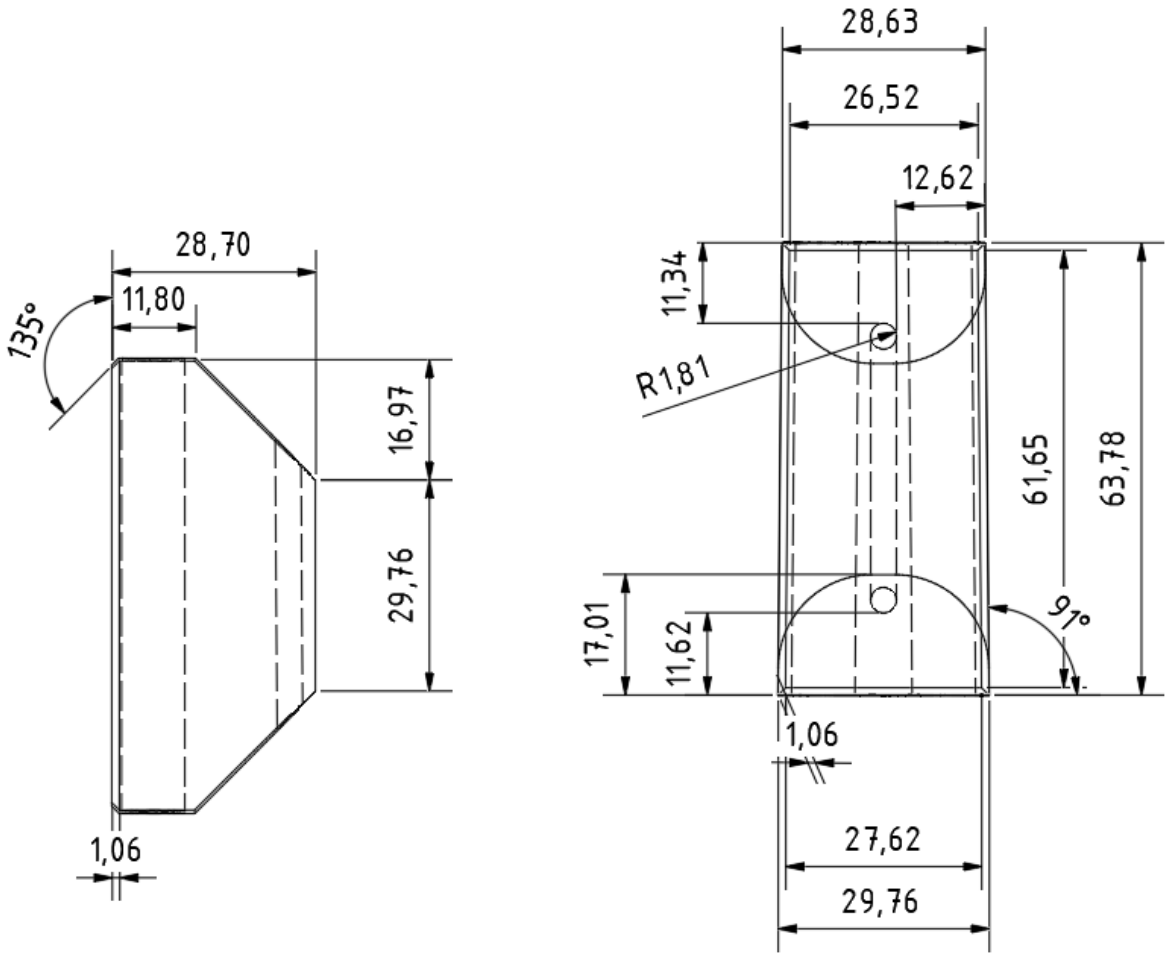

Figure 11 Finger Grundglied

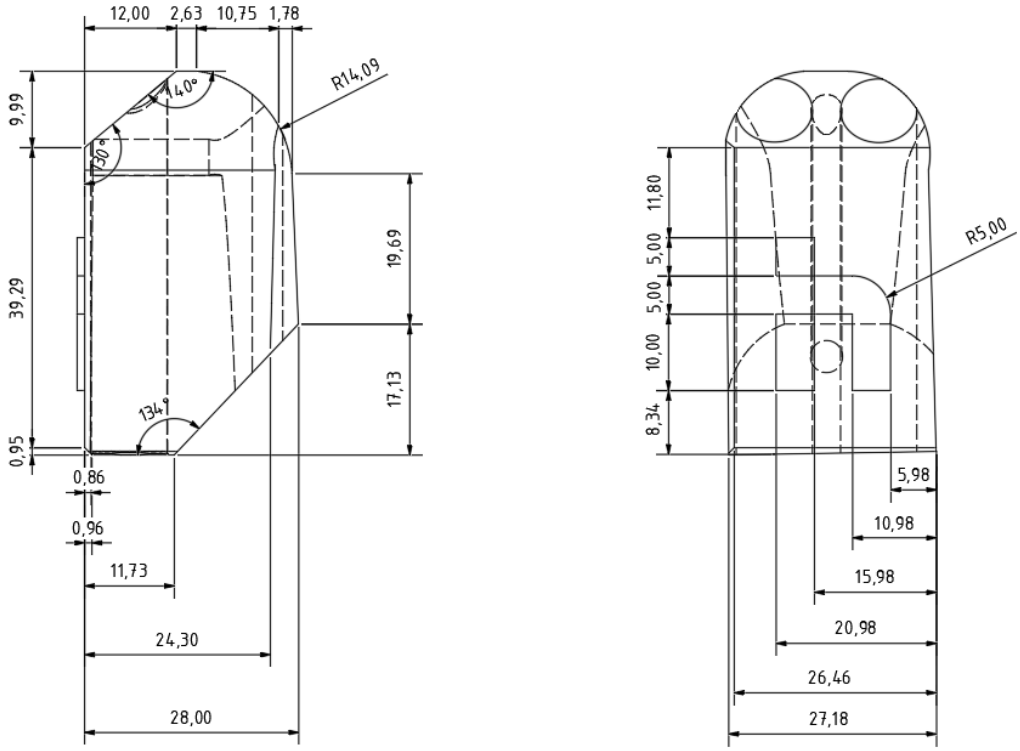

Figure 12 Endglied

Technische Zeichnungen der Einzelteile:

Finger-Segment:

- Material: PLA, 30% Infill
- Abmessungen: 40mm x 15mm x 10mm
- Features: Seil-Durchführungen, Gelenk-Aufnahmen

Servo-Halterung:

- Material: PETG (höhere Festigkeit)
- Befestigung: M3 Schrauben
- Seil-Umlenkung: Integrierte Führungsrollen

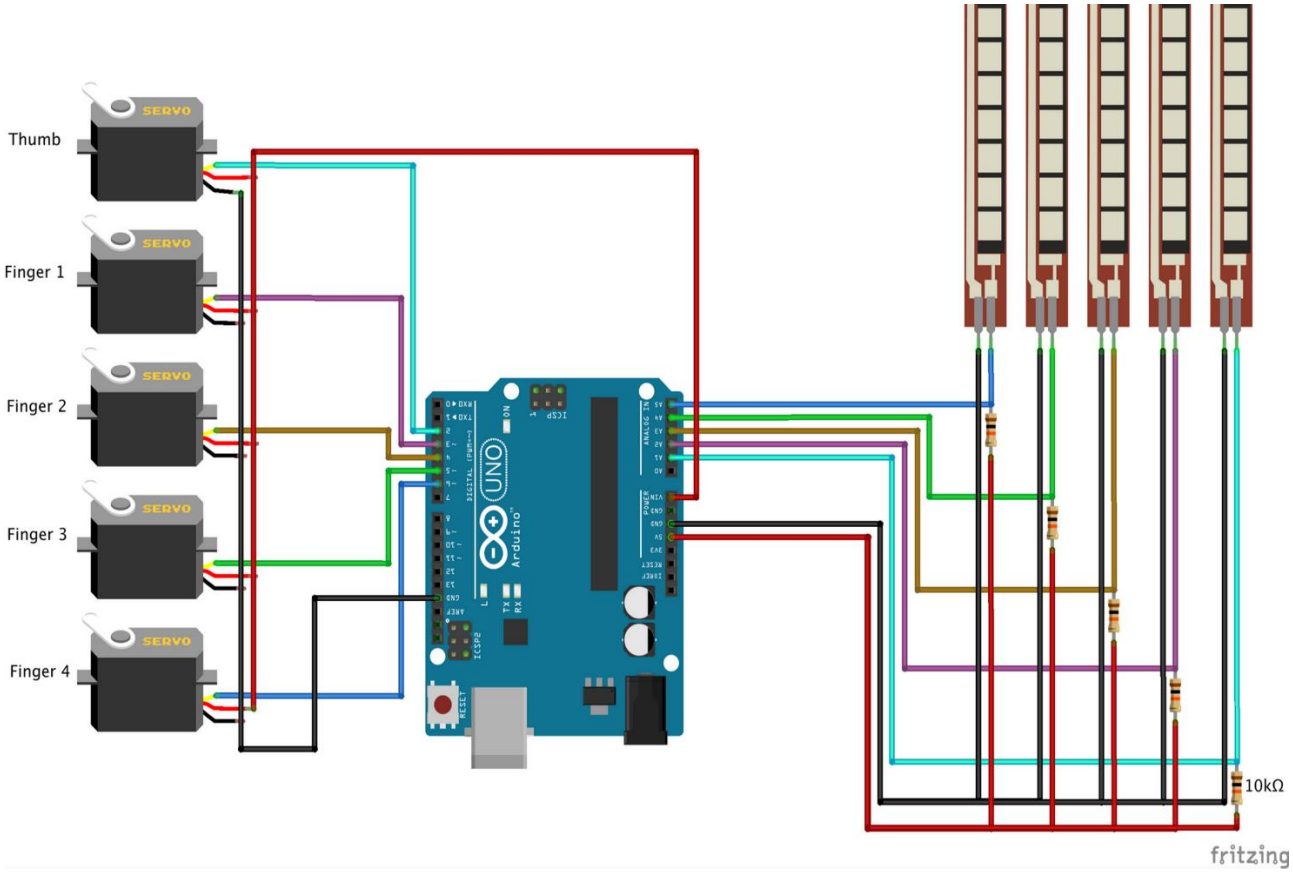

Figure 13 Schaltplan Flexsensor-Interface

4.3. Berechnungen

Mechanische Berechnungen für Sprint 1:

Servo-Kraft-Berechnung:

- Benötigtes Drehmoment pro Finger: ~2 kg·cm
- MG996R Spezifikation: 11 kg·cm (6V)
- Sicherheitsfaktor: 5x → ausreichend dimensioniert

Seil-Mechanik:

- Seilzug-Verhältnis: 1:1 (direkter Zug)
- Nylon-Angelschnur: Zugfestigkeit >10N
- Elastisches Rückstellseil: Federkonstante ~0.5 N/mm

Stromverbrauch-Berechnung:

- 5x Servo im Stillstand: 5 x 50mA = 250mA
- 5x Servo unter Last: 5 x 400mA = 2A
- Arduino UNO: ~50mA
- Gesamtverbrauch: <2.5A bei 6V

5. User Interface & Bedienung

5.1. Status-Anzeigen

LED-Status-System:

LED-Farbe	Zustand	Bedeutung	Pin
Grün	Dauerlicht	System bereit, Kalibrierung OK	D2
Grün	Blinken (1Hz)	Normaler Betrieb, Datenübertragung	D2
Gelb	Dauerlicht	Kalibrierung erforderlich	D4
Gelb	Blinken (2Hz)	Kalibrierung läuft	D4
Rot	Dauerlicht	Kritischer Fehler, System gestoppt	D7
Rot	Blinken (5Hz)	Sensor-Fehler oder Servo-Überlast	D7

5.2. Kalibrierungs-Interface

Kalibrierungs-Button (Pin D8):

- **Kurzer Druck (< 1s):** Status-Ausgabe über Serial
- **Langer Druck (> 3s):** Kalibrierungs-Routine starten
- **Sehr langer Druck (> 10s):** Kalibrierung zurücksetzen

Kalibrierungs-Ablauf:

1. Button-Press erkannt → Gelbe LED blinkt

2. "Spreizen Sie alle Finger vollständig" → 5s Erfassung

3. "Schließen Sie alle Finger vollständig" → 5s Erfassung

4. "Bewegen Sie alle Finger 3x" → 10s Erfassung

5. Validierung und EEPROM-Speicherung

6. Grüne LED → Kalibrierung erfolgreich

5.3. Benutzerinteraktion

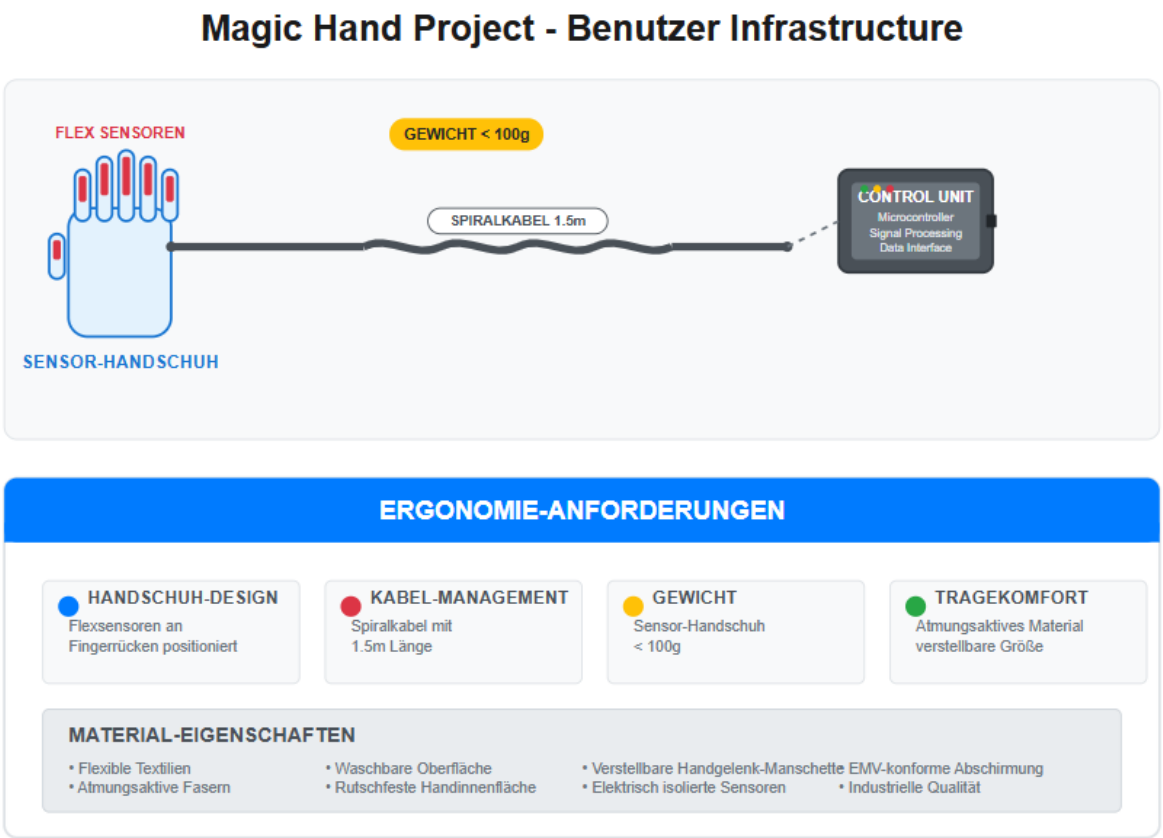

Figure 14 User Interface Layout

Ergonomie-Anforderungen:

- **Handschuh-Design:** Flexsensoren an Fingerrücken positioniert
- **Kabel-Management:** Spiralkabel mit 1.5m Länge
- **Gewicht:** Sensor-Handschuh < 100g
- **Tragekomfort:** Atmungsaktives Material, verstellbare Größe

6. Performance-Spezifikation

6.1. Latenz-Anforderungen

End-to-End Latenz-Budget:

Komponente	Ziel-Latenz	Max-Latenz	Messmethode
Sensor-Abtastung	10ms	20ms	ADC-Timing
Signal-Verarbeitung	15ms	25ms	Code-Profiling
Servo-Ansteuerung	20ms	30ms	PWM-Response
Mechanische Bewegung	50ms	80ms	Video-Analyse
Gesamt-Latenz	**95ms**	**155ms**	**Oszilloskop**

6.2. Genauigkeits-Metriken

Positionierungs-Genauigkeit:

- **Servo-Auflösung:** 180° / 1024 Steps = 0.175° pro Step
- **Ziel-Genauigkeit:** ±2° (entspricht ±11 ADC-Steps)
- **Wiederholgenauigkeit:** ±1° bei identischer Eingabe
- **Linearität:** $R^2 > 0.95$ zwischen Eingabe und Ausgabe

6.3. Zuverlässigkeits-Kriterien

Verfügbarkeits-Anforderungen:

- **MTBF (Mean Time Between Failures):** > 8 Stunden Dauerbetrieb
- **MTTR (Mean Time To Recovery):** < 30 Sekunden bei Soft-Errors
- **Erfolgsrate:** > 99% erfolgreiche Bewegungsübertragungen

7. Test & Validierung

7.1. Testkonzept Sprint 1

Test-Pyramide:

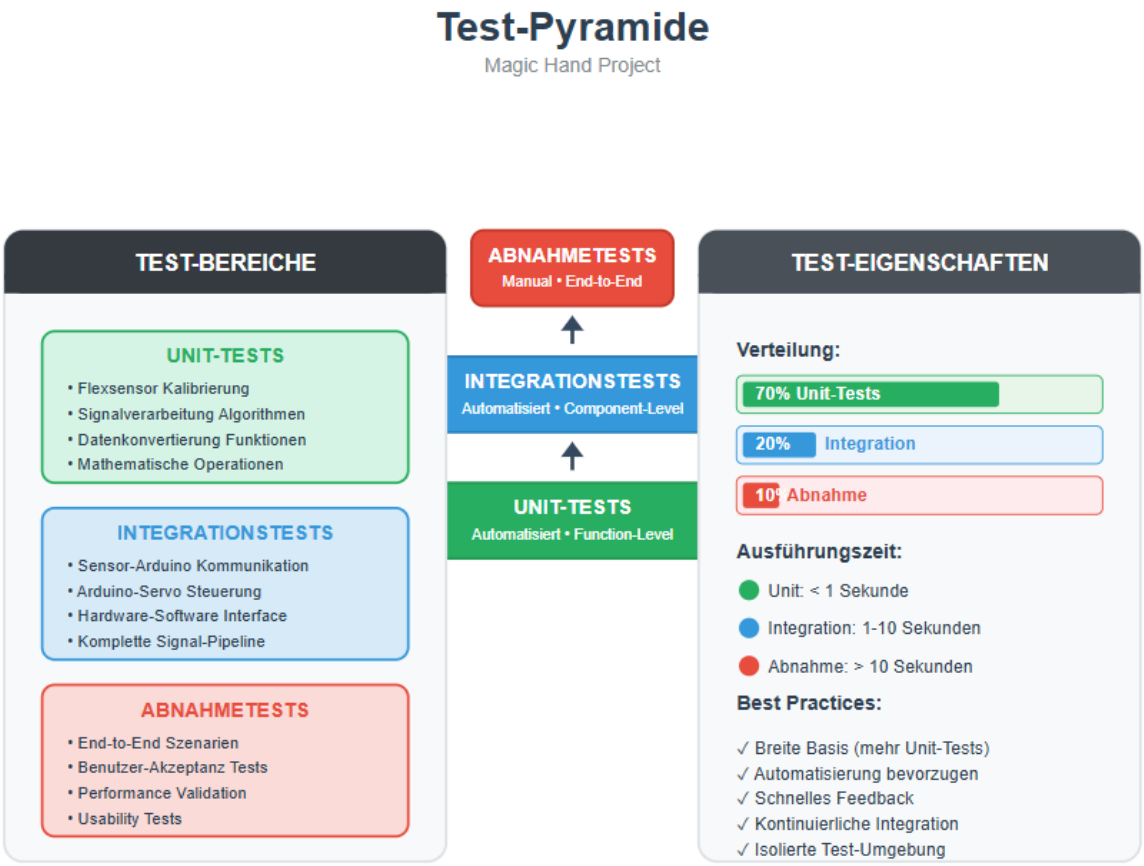

Figure 15 Test-Pyramide

7.2. Unit-Tests

Hardware Unit-Tests:

Test-ID	Komponente	Test-Beschreibung	Erwartetes Ergebnis
UT-H01	Flexsensor	ADC-Wert bei 0° Biegung	350-450 (kalibriert)
UT-H02	Flexsensor	ADC-Wert bei 90° Biegung	600-700 (kalibriert)
UT-H03	Servo MG996R	Position 0° anfahren	Servo in 0° Position
UT-H04	Servo MG996R	Position 90° anfahren	Servo in 90° Position
UT-H05	Stromversorgung	Spannung unter Last	5V ±5%, 6V ±5%

7.3. Integrationstests

System-Integration Tests:

Test-ID	Scenario	Input	Expected Output	Duration
IT-01	Einzelfinger-Bewegung	Daumen biegen 45°	Roboter-Daumen 45° ±2°	2s
IT-02	Simultane Bewegung	Alle Finger biegen	Alle Roboterfinger folgen	3s
IT-03	Hand öffnen/schließen	Faust → offene Hand	Koordinierte Bewegung	1s
IT-04	Kalibrierungs-Zyklus	Button-Press	Erfolgreiche Kalibrierung	30s

IT-05	Error-Recovery	Sensor trennen	Fallback-Verhalten	5s

7.4. Abnahmetests

Abnahme-Kriterien gemäß Pflichtenheft:

Kriterium-ID	Beschreibung	Messmethode	Sollwert	Status
AK-01	UC-1: Fingerbewegung übertragen	Video-Analyse	Alle 5 Finger beweglich	Abgeschlossen
AK-02	UC-2: Hand öffnen/schließen	Funktionstest	Koordinierte Bewegung	Abgeschlossen
AK-03	Latenz-Anforderung	Oszilloskop-Messung	< 100ms End-to-End	Abgeschlossen
AK-04	Kalibrierungs-System	Benutzer-Test	Erfolgreiche Kalibrierung	Abgeschlossen
AK-05	System-Stabilität	Dauerlauf-Test	> 30 Min ohne Fehler	Abgeschlossen

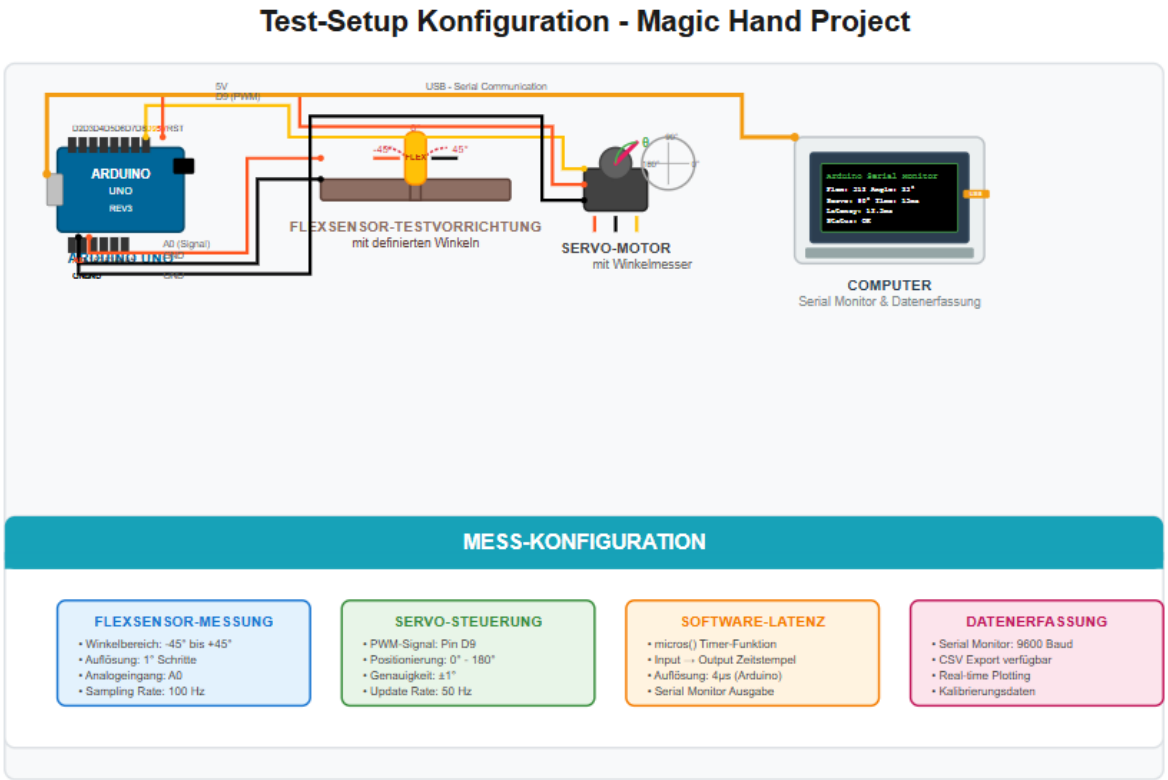

Figure 16 Test-Setup Konfiguration

8. Betrieb & Wartung

8.1. Installation und Setup

Initial-Setup Checkliste:

1. **Hardware-Vorbereitung:**

- Arduino UNO über USB anschließen
- Breadboard mit Komponenten bestücken
- Flexsensoren in Handschuh integrieren
- Servos an 3D-gedruckte Hand montieren
- Stromversorgung (6V, 3A) anschließen
- LED-Status-Anzeigen testen

2. **Software-Installation:**

- Arduino IDE installieren (≥ 2.0)
- MagicHand_Sprint1.ino öffnen
- Servo.h und EEPROM.h Libraries prüfen
- Code kompilieren (keine Fehler)
- Auf Arduino UNO flashen
- Serial Monitor öffnen (9600 baud)

3. **Ersteinrichtung:**

- System einschalten → Gelbe LED (Kalibrierung erforderlich)
- Handschuh anziehen
- Kalibrierungs-Button 3s drücken
- Kalibrierungs-Anweisungen befolgen
- Grüne LED → System bereit

8.2. Benutzerhandbuch

Täglicher Betrieb:

Schritt 1: System starten

- Stromversorgung einschalten
- Grüne LED prüfen (System bereit)
- Bei gelber LED: Neu kalibrieren
- Bei roter LED: System prüfen

Schritt 2: Handschuh anlegen

- Handschuh über Hand ziehen
- Flexsensoren auf Fingerrücken positionieren
- Kabel nicht verdrillen
- Handschuh-Verschluss schließen

Schritt 3: Betrieb

- Hand normal bewegen
- Roboterhand beobachten
- Bei Abweichungen: Kalibrierung prüfen
- Bei Problemen: LED-Status beachten

8.3. Wartung und Troubleshooting

Präventive Wartung (wöchentlich):

1. **Mechanische Inspektion:**

- Seilspannung prüfen (straff, aber nicht überspannt)
- 3D-gedruckte Teile auf Risse untersuchen
- Servo-Befestigungen auf Lockerung prüfen
- Flexsensor-Befestigung im Handschuh kontrollieren

2. **Elektrische Inspektion:**

- Kabelverbindungen auf Wackelkontakt prüfen
- Breadboard-Verbindungen nachdrücken
- Stromversorgung messen (5V ±0.2V, 6V ±0.3V)
- Flexsensor-Widerstände messen (10-35kΩ)

Diagnostische Tools:

cpp

```
void runDiagnostics() {
  Serial.println("=== MagicHand Diagnostics ===");
  // Hardware-Tests
  testPowerSupply();
  testSensorConnections();
  testServoFunctionality();
  // Software-Tests
  testCalibrationData();
  testMemoryIntegrity();
  // Performance-Analyse
  measureLatency();
  checkSystemStability();
  Serial.println("=== Diagnostics Complete ===");
```

}

9. Risikomanagement

9.1. Technische Risiken

Hardware-Risiken:

Risiko-ID	Beschreibung	Wahrscheinlichkeit	Impact	Risiko-Level
HR-01	Flexsensor-Bruch durch Überbiegung	Mittel	Hoch	Hoch
HR-02	Servo-Überlastung durch mechanische Blockade	Niedrig	Mittel	Mittel
HR-03	3D-Druck-Qualität unzureichend	Mittel	Mittel	Mittel
HR-04	Stromversorgung instabil	Niedrig	Hoch	Mittel
HR-05	Kabelbruch im Handschuh	Hoch	Mittel	Hoch

Software-Risiken:

Risiko-ID	Beschreibung	Wahrscheinlichkeit	Impact	Risiko-Level
SR-01	Kalibrierung schlägt fehl	Mittel	Hoch	Hoch
SR-02	Memory-Overflow bei langen Sessions	Niedrig	Mittel	Niedrig
SR-03	ADC-Rauschen verfälscht Werte	Mittel	Mittel	Mittel
SR-04	EEPROM-Korruption	Niedrig	Hoch	Mittel

SR-05	Timing-Probleme bei hoher Last	Niedrig	Niedrig	Niedrig

9.2. Mitigation-Strategien

Hardware-Mitigation:

HR-01: Flexsensor-Schutz

```cpp
// Software-Schutz vor Sensor-Überlastung
bool validateSensorRange(int sensorValue) {
    if(sensorValue < MIN_SAFE_VALUE || sensorValue > MAX_SAFE_VALUE) {
        Serial.println("WARNING: Sensor out of safe range!");
        return false;
    }
    return true;
}
```

Software-Mitigation:

SR-01: Robuste Kalibrierung

```cpp
bool performRobustCalibration() {
    int attempts = 0;
    const int MAX_ATTEMPTS = 3;

    while(attempts < MAX_ATTEMPTS) {
        if(runCalibrationSequence()) {
            if(validateCalibrationQuality() > 0.8) {
                return true; // Erfolgreiche Kalibrierung
            }
        }
        attempts++;
        Serial.println("Kalibrierung Versuch " + String(attempts));
    }
```

```
return false; // Kalibrierung fehlgeschlagen
```

9.3. Contingency-Pläne

Plan A: Sensor-Ausfall

1. Defekten Sensor identifizieren
2. Fallback-Modus aktivieren (4-Finger-Betrieb)
3. Benutzer informieren (LED + Serial)
4. Sensor-Austausch vorbereiten

Plan B: Servo-Ausfall

1. Servo-Überlastung erkennen
2. Betroffenen Servo deaktivieren
3. Mechanische Position sichern
4. Weiterbetrieb mit reduzierten Fingern

Plan C: Kalibrierungs-Fehler

1. Automatische Fehlerdiagnose
2. Fallback auf Werks-Kalibrierung
3. Benutzer über eingeschränkte Funktion informieren
4. Manuelle Re-Kalibrierung anbieten

10. Sprint-Deliverables

10.1. Lieferumfang Sprint 1

Hardware-Deliverables:

- 1x Arduino UNO mit MagicHand Software
- 5x Flexsensor 2.2" (kalibriert und getestet)
- 5x Servo MG996R (konfiguriert)

- 1x 3D-gedruckte Roboterhand (komplette Mechanik)
- 1x Sensor-Handschuh (integrierte Flexsensoren)
- 1x Prototyping-Board mit allen Verbindungen
- 1x Stromversorgung (5V + 6V)
- Komplett verkabeltes System (plug-and-play)

Software-Deliverables:

- MagicHand_Sprint1.ino (Hauptprogramm)
- Sensor-Manager Module
- Servo-Controller Module
- Kalibrierungs-System
- Error-Handling und Recovery
- Test-Suite (Unit + Integration)
- Diagnostics-Tools

Dokumentation:

- Technische Spezifikation (dieses Dokument)
- Benutzerhandbuch (Kurzfassung)
- Test-Protokolle und -Ergebnisse
- Schaltpläne und Pin-Belegung
- 3D-CAD-Dateien (.ipt, .stl)
- Installationsanleitung
- Troubleshooting-Guide

10.2. Übergabe-Kriterien

Funktionale Kriterien:

- UC-1: Fingerbewegung übertragen - vollständig implementiert
- UC-2: Hand öffnen/schließen - vollständig implementiert
- Kalibrierungs-System - funktional und benutzerfreundlich

- Error-Handling - robuste Fehlerbehandlung
- Performance: Latenz < 100ms, Genauigkeit ±2°
- Stabilität: 30 Minuten Dauerbetrieb ohne Fehler

Qualitäts-Kriterien:

- Code-Qualität: Kommentiert, strukturiert, wartbar
- Hardware-Qualität: Alle Verbindungen stabil, mechanisch robust
- Test-Coverage: >90% der Funktionen getestet
- Dokumentation: Vollständig und verständlich
- Benutzerfreundlichkeit: Intuitive Bedienung
- Erweiterbarkeit: Vorbereitung für Sprint 2 erkennbar

10.3. Sprint 2 Vorbereitung

Technische Vorbereitung:

- **Pin-Reservierung:** D11-D13 für SPI (NRF24L01+)
- **Software-Architektur:** Kommunikations-Layer vorbereitet
- **Hardware-Layout:** Platz für drahtlose Module
- **Stromversorgung:** Dimensionierung für zusätzliche Module

Architektur-Erweiterungen:

cpp

```
// Vorbereitung für drahtlose Kommunikation

class CommunicationManager {

private:

  bool wirelessMode = false;

public:
```

```
    virtual void sendSensorData(SensorData data) = 0;

    virtual bool receiveCommands() = 0;

    // Sprint 1: Kabelgebunden

    void enableWiredMode() { wirelessMode = false; }

    // Sprint 2: Drahtlos

    void enableWirelessMode() { wirelessMode = true; }

};
```

11. Offene Fragen

#	Issue	Status	Owner	Deadline
1	Optimale Flexsensor-Positionierung am Handschuh	Offen	Projektteam	10.06.2025
2	3D-Druck-Parameter für maximale Stabilität	In Bearbeitung	Ali M.	08.06.2025
3	Kalibrierungsverfahren für verschiedene Benutzer	Offen	Taoufik	12.06.2025
4	Seil-Spannung Optimierung	In Bearbeitung	Lukas	09.06.2025
5	Integration von Status-Feedback (LEDs)	Offen	Farhad	11.06.2025
6	Performance-Optimierung für <100ms Latenz	Offen	Projektteam	15.06.2025
7	Handschuh-Material für Langzeit-Tragekomfort	Offen	Ali M.	12.06.2025
8	EEPROM-Layout für Kalibrierungsdaten	In Bearbeitung	Taoufik	10.06.2025
9	Error-Recovery Strategien definieren	Offen	Lukas	13.06.2025
10	Sprint 2 Hardware-Interface spezifizieren	Offen	Farhad	16.06.2025

12. Modul Abhängigkeiten

Abhängigkeiten zu anderen Modulen:

Software-Abhängigkeiten:

- Arduino Core Framework (ATmega328P)
- Servo.h Library (Standard Arduino)
- EEPROM.h Library (Kalibrierungsdaten-Persistierung)
- SPI.h Library (Vorbereitung für Sprint 2 - NRF24L01+)

Hardware-Abhängigkeiten:

- Externe Stromversorgung für Servo-Betrieb (6V, 3A)
- Breadboard-Prototyping-Umgebung
- 3D-Drucker für mechanische Komponenten (PLA/PETG Material)
- Lötstation für finale Verbindungen (optional für Sprint 1)

Entwicklungs-Abhängigkeiten:

- Arduino IDE ≥ 2.0 (Cross-Platform Entwicklung)
- Autodesk Inventor (3D-CAD für mechanische Komponenten)
- Serial Monitor für Debugging und Kalibrierung
- Oszilloskop für Performance-Messungen (optional)

Sprint 2 Vorbereitung:

- Pin-Reservierung für NRF24L01+ Module (D11-D13, SPI)
- Software-Architektur vorbereitet für drahtlose Erweiterung
- Mechanische Befestigungspunkte für zusätzliche Hardware
- Strombudget-Reservierung für drahtlose Module

Externe Abhängigkeiten:

- Keine kritischen externen Services benötigt
- Alle Komponenten lokal verfügbar und testbar
- Offline-Entwicklung vollständig möglich
- Internet nur für Library-Updates erforderlich

Risiko-Abhängigkeiten:

- 3D-Drucker-Verfügbarkeit für Prototyping
- Komponenten-Verfügbarkeit (Flexsensoren, Servos)
- HTW-Labor-Zugang für Tests und Integration
- Projektteam-Verfügbarkeit für koordinierte Entwicklung

13. Geplante Erweiterungen (Sprint 2+3)

Hinweis: Diese Spezifikation ist modular aufgebaut für nahtlose Erweiterung:

Sprint 2 Integration:

- NRF24L01+ Module-Slots vorgesehen (Pin D11-D13)
- Software-Module für drahtlose Kommunikation vorbereitet
- Gestenerkennung als zusätzliche Komponente (UC-4)
- Batterie-Management für mobile Sensoreinheit

Sprint 3 Integration:

- Objektinteraktions-Module (UC-5, UC-6, UC-7)
- Erweiterte Sensorik-Schnittstellen
- Greifkraft-Optimierungen
- Objekterkennung und -klassifikation